TransferIa · Documentos de cambios

Videos, lotes y configuraciones

Entrega 9 · Detener y continuar, fragmentos, carpetas y configuraciones

Commit: `672f82a` "Nuevas funcionalidades desde mejorando angulos"

Fecha: 26 de octubre de 2026

Documento generado: 27 de octubre de 2026

Pruebas: caras generadas por IA, en CPU

Contenido

1. Resumen de la entrega 2
2. Videos: el problema 2
3. La solución 3
3.1 Directo a ffmpeg 3
3.2 Un formato que aguanta cortes....3
3.3 Las partes y el progreso 3
3.4 Probado: un corte brusco 3
4. Solo un fragmento 4
5. Carpeta de imágenes como destino 5
6. Configuraciones guardadas....6
7. Cómo se comprobó....6
8. Lo que aprendimos 6

Si el índice aparece vacío, en Word haz clic derecho sobre él → Actualizar campos.

1. Resumen de la entrega

> **Nota:** Todo este documento sale del commit `672f82a` del 26 de octubre, que juntó unas 25 funciones nuevas en 26 archivos (5748 líneas). Se repartió en las entregas 4 a 9 por tema. Las pruebas se hicieron con una copia de esa versión (`git archive`) y caras **generadas por IA**; los números marcados "(README)" son mediciones de la sesión de desarrollo que quedaron escritas en el README y no se volvieron a medir.

Cambio	CLI	Qué gana el usuario
Detener y continuar, aunque se corte la luz	`--resume`, `--restart`	No perder horas de trabajo
Sin imágenes temporales y con poca RAM	—	Videos largos sin llenar el disco ni la memoria
Formato y códec del video temporal	`--container`, `--video-codec`, `--crf`	Aguantar cortes; usar la tarjeta de video
Solo un fragmento	`--start`, `--end`	Procesar solo los segundos que interesan
Carpeta de imágenes como destino	`--target carpeta`	Procesar muchas fotos de una vez
Configuraciones guardadas	`--config`, `--save-config`	Guardar todas las opciones con un nombre

2. Videos: el problema

- La ventana guardaba **cada frame como JPG** en `temp_frames/` y al final los volvía a comprimir: llenaba el disco y perdía calidad dos veces.
- Si el proceso se cortaba (apagón, cierre), el MP4 temporal quedaba **inservible**: un MP4 normal guarda su índice al final.
- No había forma de continuar: había que empezar de cero.

3. La solución

3.1 Directo a ffmpeg

Cada frame terminado va **directo a ffmpeg por una tubería** (`Escritor_Video`). En memoria solo están los frames que se están procesando. ffmpeg comprime en H.264 una sola vez.

3.2 Un formato que aguanta cortes

El temporal es **MKV** (o MP4 fragmentado), que guarda el video en bloques: si se corta, se reproduce hasta el corte. Además se graba **sin B-frames** y con un fotograma clave cada 2 s, para que cada paquete sea un frame en orden y se pueda recortar exacto.

3.3 Las partes y el progreso

```
salida.mp4.partes/
├── parte_001.mkv      lo procesado en la primera sesión
├── parte_002.mkv      una parte nueva cada vez que se continúa
└── progreso.json      video, opciones (firma), frames hechos; se guarda cada 2 s
```

Al continuar: se cuentan los frames que **de verdad** tiene la última parte, se descartan los 2 últimos por si quedaron a medias, y se sigue desde ahí. Al terminar, las partes se unen **sin recomprimir** y se agrega el audio.

3.4 Probado: un corte brusco

Se procesó un video de 100 frames y se **mató el proceso** desde Windows 6 s después de que empezó a escribir, como un apagón (sin dejarle cerrar nada):

```
> dir salida\e9_corte.mp4.partes   (despues del corte)
parte_001.mkv     99 815 bytes   (22 frames reales)
progreso.json      1 109 bytes

> progreso.json (extracto)
 "total": 100, "fps": 25.0,
 "partes": [ { "archivo": "parte_001.mkv", "frames": 15, "cerrada": false } ]
```

Figura 1. *Lo que quedó en la carpeta de partes y en progreso.json*

Al volver a ejecutar **sin** `--resume`, avisa en lugar de borrar lo hecho; y si cambia una opción, **no deja continuar** para que el video no cambie de estilo a la mitad:

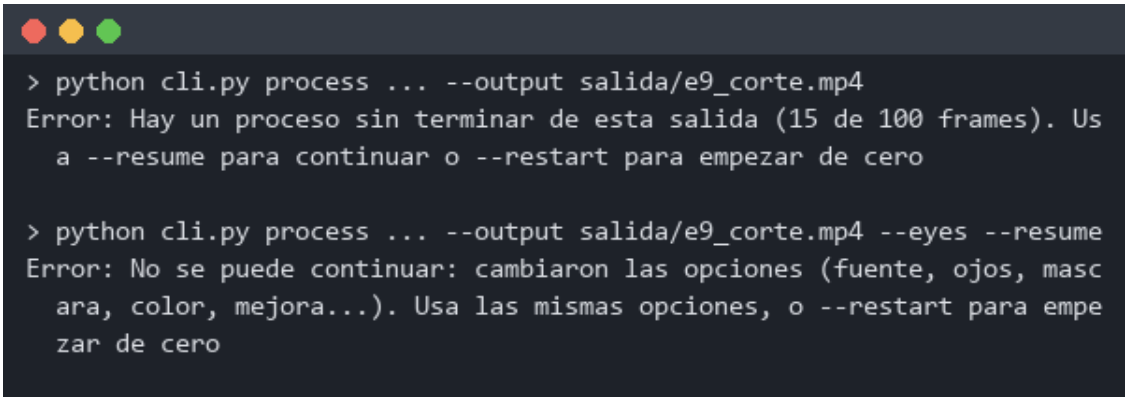


Figura 2. *Sin --resume, y con --resume pero agregando --eyes*

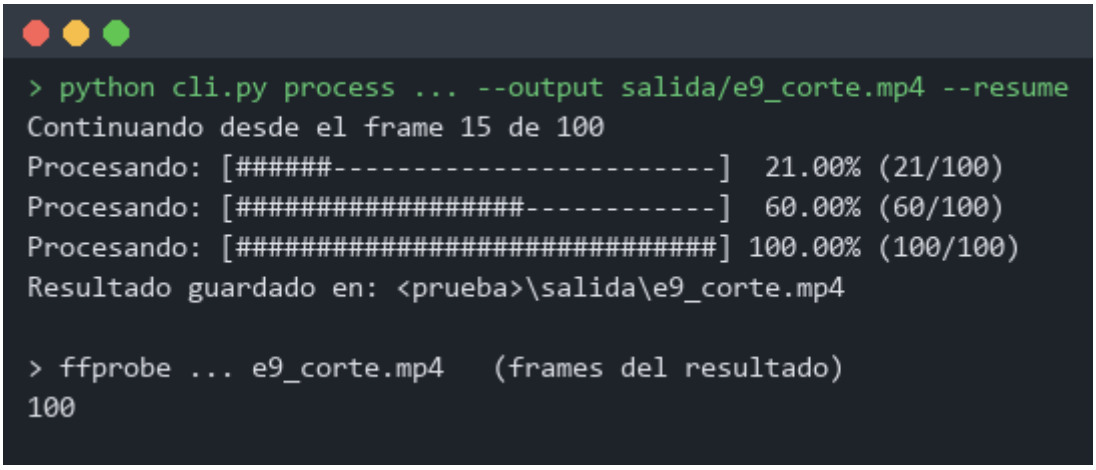


Figura 3. *Con --resume y las mismas opciones: continúa y termina*

Resultado: el video final tiene **100 de 100 fotogramas** (contados con ffprobe). La parte cortada tenía 22 fotogramas reales, aunque `progreso.json` decía 15 (se guarda cada 2 s): al continuar se recortó a 22 − 2 = **20** y se siguió desde ahí (la barra arranca en 21/100).

> **Nota:** El mensaje dice "Continuando desde el frame 15" porque muestra el número de `progreso.json` **antes** de revisar el archivo; en realidad continúa desde el 20. Es solo un texto inexacto (el video sale bien) y sigue así en la versión actual (`cli.py`, línea 580).

4. Solo un fragmento

Con `--start` y `--end` (en segundos, `m:ss` o `h:mm:ss`) se procesa solo ese tramo, con su audio.

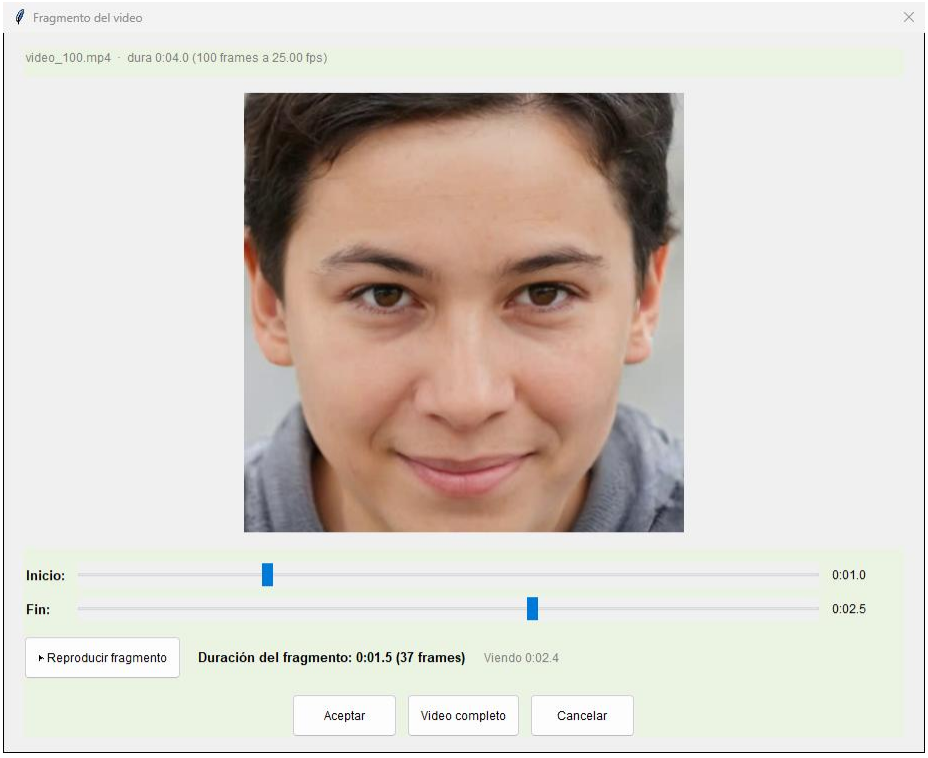


Figura 4. *Ventana "Fragmento del video": inicio, fin, duración y reproducir*

Para llegar al inicio: si está cerca (hasta 300 frames) lee y descarta; si está lejos, salta al fotograma clave anterior y avanza. Si el salto no queda exacto, lee desde el principio: **el fragmento siempre empieza en el frame justo**.

Probado con `--start 1.0 --end 2.5` en un video de 25 fps: la CLI anunció "1.00s a 2.48s (1.48s, 37 frames)" y el resultado tiene **37 fotogramas** (ffprobe).

5. Carpeta de imágenes como destino

Si `--target` es una carpeta, cada imagen se procesa con las mismas opciones y se guarda **con su mismo nombre** en `<salida>/<carpeta>_output/`. Se procesan en paralelo según los hilos; si una falla, se avisa al final y se siguen las demás.

```
> python cli.py process --source fuente.jpg --target carpeta_fotos --output salida
3 imagen(es) -> <prueba>\salida\carpeta_fotos_output
3 de 3 imagen(es) guardada(s)
Resultado guardado en: <prueba>\salida\carpeta_fotos_output
Imagenes: [##############################] 100.00% (3/3)

> dir salida\carpeta_fotos_output
ana.jpg
grupo.jpg
luis.jpg
```

Figura 5. *Una carpeta con tres fotos de IA*

6. Configuraciones guardadas

Todas las opciones se guardan con un nombre en `configuraciones/<nombre>.json`, y sirven para la ventana y la CLI. Una configuración dice **cómo** procesar, no **qué**: no guarda la fuente, el destino ni el fragmento.

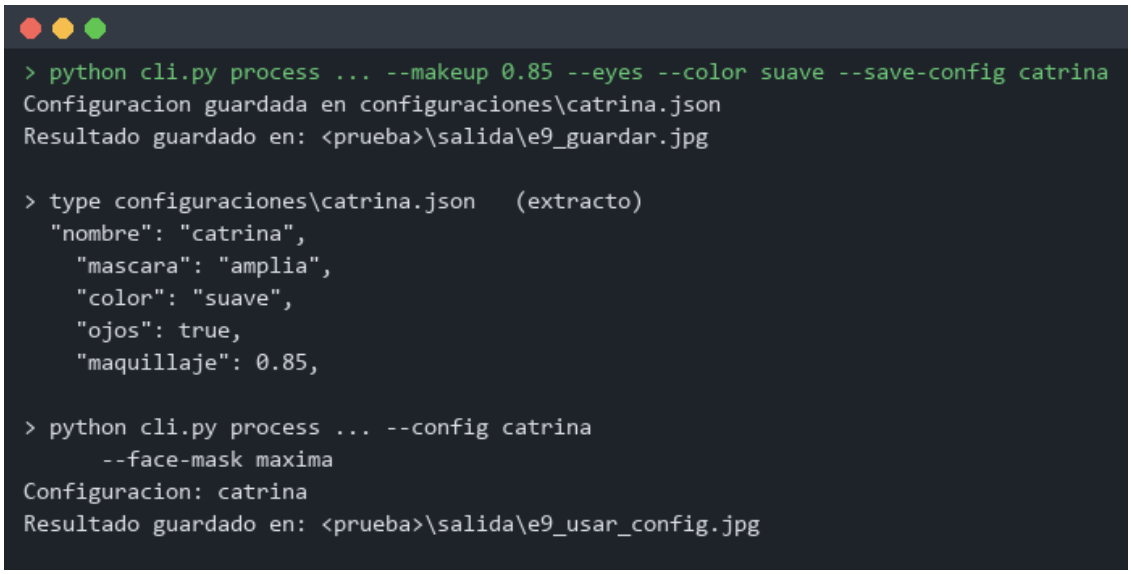


Figura 6. *Guardar "catrina" con --save-config y el archivo que se creó*

Al cargar, cada valor se valida: si uno es inválido o falta, esa opción se deja como estaba y se avisa; las demás se cargan. Así una configuración de una versión anterior sigue sirviendo. En la CLI, lo escrito en el comando tiene prioridad (`--config catrina --face-mask maxima`).

7. Cómo se comprobó

- Corte brusco real (proceso terminado a la fuerza con Stop-Process) y continuación con `--resume`; se contaron los frames del resultado con ffprobe.
- Fragmento de 1,0 s a 2,5 s de un video de 25 fps; se contaron sus frames.
- Carpeta de 3 imágenes y una configuración guardada y reutilizada.

8. Lo que aprendimos

- Para sobrevivir a un corte: formato que se pueda leer a medias + progreso guardado seguido + descartar lo último dudoso.
- Una "firma" de las opciones impide mezclar estilos en un mismo video.
- Tuberías en vez de archivos temporales: menos disco, menos RAM y una sola compresión.
- Validar valor por valor hace que los archivos de configuración viejos sigan funcionando.